

Mock Design: A Payment Gateway

Phase 4 · Interview-Ready · Estimated time: 2 hours

1. Concept From First Principles: Applying the Framework

A payment gateway (Razorpay/PhonePe/Stripe-style) is arguably the highest-stakes problem in this Phase: unlike a feed being briefly stale or a cache missing occasionally, a payment system getting correctness wrong means real money lost or double-charged. This chapter is intentionally the deepest application yet of Phase 3 Chapter 16's capstone ideas — event sourcing, idempotency, and ledger design — now run through the interview framework as a standalone problem rather than a recap.

Step 1 — Clarify requirements

- Functional: accept a payment request, route it to the appropriate payment method/bank/network, return success/failure, and record an immutable, auditable transaction trail.
- Non-functional: correctness is absolute — no lost transactions, no double charges, exact reconciliation with banks/networks must always be possible after the fact.
- Clarify scope: is this a gateway that itself holds funds (like a wallet) or purely a pass-through processor to banks/card networks? This materially changes the ledger design.

Step 2 — Estimate scale

Assume 50 million transactions/day (~580/second average, several times that at peak around festival sales or salary-day bill payments). The number that matters isn't raw throughput here — it's modest compared to earlier chapters — but the **cost of a single incorrect transaction**, which is why this system's design optimizes for correctness and auditability far more aggressively than for raw throughput.

Step 3 — API

POST /payments {idempotencyKey, amount, currency, source, destination} -> {paymentId, status}. The idempotency key isn't optional or an afterthought here — it's a required field on the very first API call, since this is the single most important correctness mechanism in the entire system, worth stating explicitly at the API-design step rather than only in the deep dive.

Step 4 — High-level design

Client → API Gateway → Payment Service, which orchestrates: (1) idempotency check, (2) fraud/risk check, (3) route to the appropriate payment processor/bank, (4) record the outcome in an immutable ledger. Given the correctness stakes and the value of a full audit trail, the ledger's write path uses **event sourcing** (Phase 3, Ch 8): every state change ("PaymentInitiated," "PaymentAuthorized," "PaymentCaptured," "PaymentFailed") is an immutable, appended event, never an in-place update — directly reusing this program's established capstone pattern from Phase 3 Chapter 16, now as the entire focus of a standalone question rather than one piece of a larger system.

Step 5 — Deep dive: idempotency, the double-entry ledger, and reconciliation

Idempotency, precisely: a client-generated idempotency key is stored alongside the payment attempt before any external call is made. If the same key arrives again (client retry after a timeout, where the original request may have actually succeeded despite the client never seeing a response), the system returns the stored original result instead of re-attempting the charge — this exact scenario, ambiguous network failure after a successful server-side charge, is the single most common real cause of duplicate-charge bugs in production payment systems, which is why interviewers probe it so specifically.

Double-entry ledger: every money movement is recorded as a matched pair of debit and credit entries that must always sum to zero across the full ledger (money debited from one account is always credited to another, never created or destroyed) — a discipline borrowed directly from traditional accounting, and the standard way production payment systems make bugs detectable, since a ledger that doesn't balance is immediate, unambiguous proof something is wrong, long before a human would otherwise notice. This ledger is append-only, consistent with the event-sourced design.

Reconciliation: because payment gateways route to external banks/networks, occasional divergence between what the gateway's own records show and what the bank's records show is a genuine operational reality (a network timeout on the bank's side, for instance). A batch reconciliation job (Phase 3, Ch 7) periodically compares the gateway's ledger against bank-provided settlement files and flags mismatches for investigation — naming this explicitly, unprompted, is a strong senior-level signal, since it shows awareness that correctness within your own system isn't sufficient when the system depends on external parties' records too.

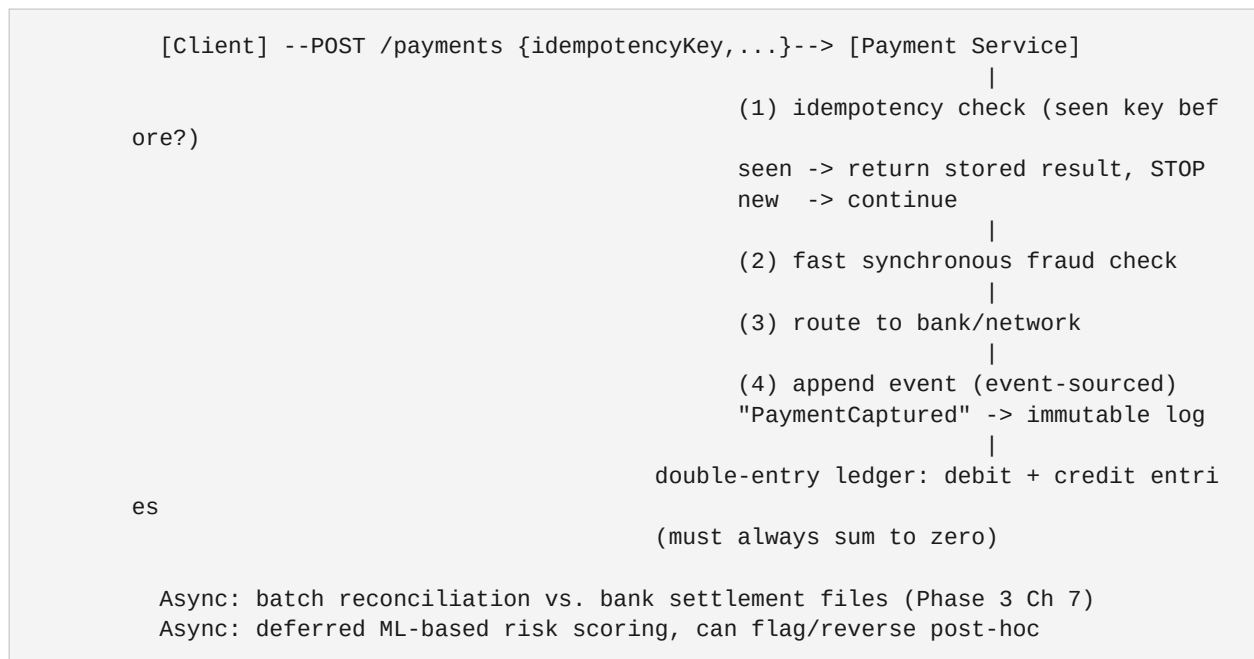
Step 6 — Trade-offs and wrap-up

Volunteer: fraud/risk checks add latency to the critical payment path — discuss running fast, cheap heuristic checks synchronously (blocking the payment) while deferring more expensive machine-learning-based risk scoring to an asynchronous post-payment review that can flag and reverse a transaction after the fact if needed, rather than making every payment wait on the slowest possible check.

2. Real-World Example / Case Study

Razorpay's and PhonePe's publicly discussed engineering blogs both describe double-entry ledger design and reconciliation against bank settlement files as core, non-negotiable parts of their payment infrastructure — this is precisely why fintech-adjacent Indian companies were noted in Chapter 2 as probing correctness and failure-mode thinking harder than almost any other interview topic, since this is quite literally the production reality their engineers live in daily.

3. Diagram



4. Trade-offs Table

Decision	Alternative	Why this choice
Event sourcing for the payment ledger	In-place status updates	Immutable audit trail; matches Phase 3 Ch 8/16's established pattern
Mandatory idempotency key on every request	Optional/best-effort dedup	Prevents the single most common real cause of duplicate charges
Double-entry ledger (debit/credit pairs)	Single balance field per account	Self-detecting correctness: an unbalanced ledger is immediate proof of a bug
Sync fast fraud check + async deep risk scoring	Full ML risk scoring synchronously	Avoids adding heavy latency to every payment on the critical path

5. Common Interview Questions

Q1: "How do you prevent a client's retried request from charging the customer twice?"

Model approach: Explain the idempotency key mechanism precisely, including the specific ambiguous-timeout scenario it solves — a retry after a successful-but-unacknowledged charge returning the stored original result rather than processing a new one.

Q2: "Why use a double-entry ledger instead of just storing a balance per account?"

Model approach: Name the self-detecting correctness property directly — every debit has a matching credit, so the ledger sums to zero when correct, making bugs immediately, unambiguously visible rather than silently corrupting a single balance field.

Q3: "What do you do if your records and the bank's records disagree?"

Model approach: Describe the batch reconciliation job against bank settlement files directly, and frame it as an accepted operational reality of integrating with external parties, not a design flaw to be engineered away entirely.

6. Practice Exercises

- 1 Run the full six-step framework for "design a payment gateway," out loud, with a timer.
- 2 Design the exact schema for a double-entry ledger table, and write a query that verifies the ledger sums to zero for a given account.
- 3 Design the reconciliation job's logic: given the gateway's ledger and a bank settlement file, how do you efficiently identify mismatches?

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Payment Processing System Design Explained: Idempotency, Security High Level Design Interview"	Directly covers idempotency and correctness for payment systems.
English	"Design Stripe: System Design Interview (Stripe & Amazon Offers)"	Full mock-interview-style walkthrough of a real payment gateway design.
Hindi	"System Design 17: Design Payment Gateway Payment System Stripe Paypal Razorpay HLD LLD"	Hindi/Hinglish walkthrough referencing Razorpay directly, relevant to Indian-market prep.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering payment and ledger design as part of the broader curriculum.

8. Key Takeaways

- Payment systems optimize for correctness and auditability far more aggressively than raw throughput, unlike most earlier mock-design chapters.
- Idempotency keys are mandatory, not optional, and solve a specific, common real-world duplicate-charge scenario.
- Event sourcing gives the payment ledger a full, immutable audit trail, directly reusing Phase 3 Chapter 16's capstone pattern.
- A double-entry ledger is self-detecting: an unbalanced ledger is immediate, unambiguous proof of a bug.
- Reconciliation against external bank/network settlement files is a necessary, accepted operational reality, not an edge case.
- Fast synchronous fraud checks plus deferred async deep risk scoring balances latency against thoroughness.
- This chapter is the deepest standalone application yet of Phase 3's event sourcing and idempotency material.